

AWS Deployment Guide

Deploys the pManager serverless backend: **API Gateway (HTTP API) → Lambda → DynamoDB**.

Prerequisites

Tool	Purpose	Install
AWS CLI v2	AWS authentication	<code>brew install awscli</code>
AWS SAM CLI	Build & deploy	<code>brew install aws-sam-cli</code>
Node.js 20+	Lambda runtime dependency install	<code>brew install node</code>
Docker (optional)	SAM local testing	<code>brew install --cask docker</code>

Step 1 — Configure AWS credentials

```
aws configure
```

Enter your **Access Key ID**, **Secret Access Key**, default region (`ca-central-1`), and output format (`json`).

Verify access:

```
aws sts get-caller-identity
```

Step 2 — Install Lambda dependencies

From the repo root:

```
cd aws/lambda
npm install
cd ../..
```

Step 3 — Build the SAM application

```
cd aws
sam build
```

SAM compiles the Lambda code and packages it under `.aws-sam/build/`.

Step 4 — Deploy (first time)

Run the guided deploy to create `samconfig.toml` (already committed with defaults):

```
sam deploy --guided
```

Prompted values — press **Enter** to accept the defaults shown in brackets:

Prompt	Default / recommended value
Stack Name	<code>pmanager</code>
AWS Region	<code>ca-central-1</code>
<code>AllowedOrigin</code>	<code>*</code> (for local dev) or your frontend URL in production
Confirm changeset before deploy	<code>Y</code>
Allow SAM to create IAM roles	<code>Y</code>
Save arguments to <code>samconfig.toml</code>	<code>Y</code>

Production tip: set `AllowedOrigin` to the exact URL serving your frontend (e.g. `https://mysite.example.com`) to restrict CORS.

Step 5 — Note the stack outputs

After a successful deploy SAM prints the stack outputs:

Key	<code>ApiUrl</code>
Description	API base URL

Value	https://<api-id>.execute-api.ca-central-1.amazonaws.com
Key	FunctionName
Value	pmanager-api-pmanager
Key	TableName
Value	pmanager-pmanager

Copy the **ApiUrl** value — you will need it in the next step.

Step 6 — Update frontend configuration

Replace the `apiBase` value in **both** frontend config files:

`manager/config.js`

```
window.PM_CONFIG = {  
  apiBase: "https://<api-id>.execute-api.ca-central-1.amazonaws.com",  
  useLocalCache: true,  
};
```

`vault/config.js`

```
window.PM_CONFIG = {  
  apiBase: "https://<api-id>.execute-api.ca-central-1.amazonaws.com",  
  useLocalCache: true,  
};
```

Subsequent deployments

Once `samconfig.toml` exists, deploy without the wizard:

```
cd aws  
sam build && sam deploy
```

Useful commands

Command	Description
<code>sam build</code>	Re-package Lambda code
<code>sam deploy</code>	Deploy / update the CloudFormation stack
<code>sam logs -n pmanager-api-pmanager --tail</code>	Stream live Lambda logs
<code>sam local start-api</code>	Run API locally via Docker
<code>aws cloudformation describe-stacks --stack-name pmanager</code>	View stack status
<code>aws cloudformation delete-stack --stack-name pmanager</code>	Delete stack (DynamoDB table is retained)

Stack resources created

Resource	AWS Type	Notes
<code>pmanager-pmanager</code>	DynamoDB Table	<code>DeletionPolicy: Retain</code> — survives stack deletion
<code>pmanager-api-pmanager</code>	Lambda Function	Node.js 24.x, arm64 (Graviton2)
HTTP API	API Gateway v2	Routes <code>/api/{proxy+}</code> to Lambda
Execution Role	IAM Role	DynamoDB CRUD permissions only

Tear-down

```
aws cloudformation delete-stack --stack-name pmanager
```

The DynamoDB table (`pmanager-pmanager`) is **not** deleted because of `DeletionPolicy: Retain` . Delete it manually in the AWS Console if you no longer need the data.